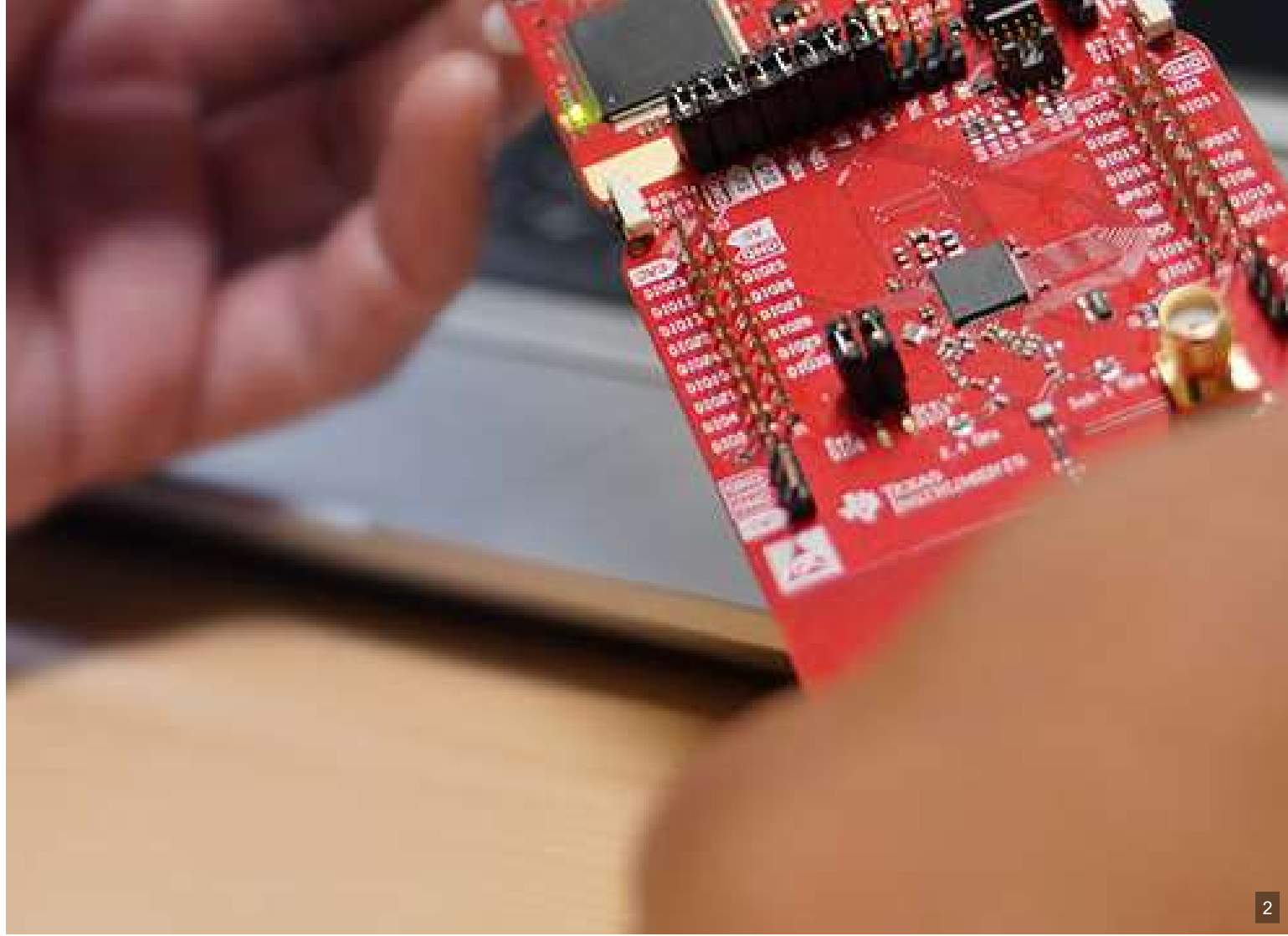
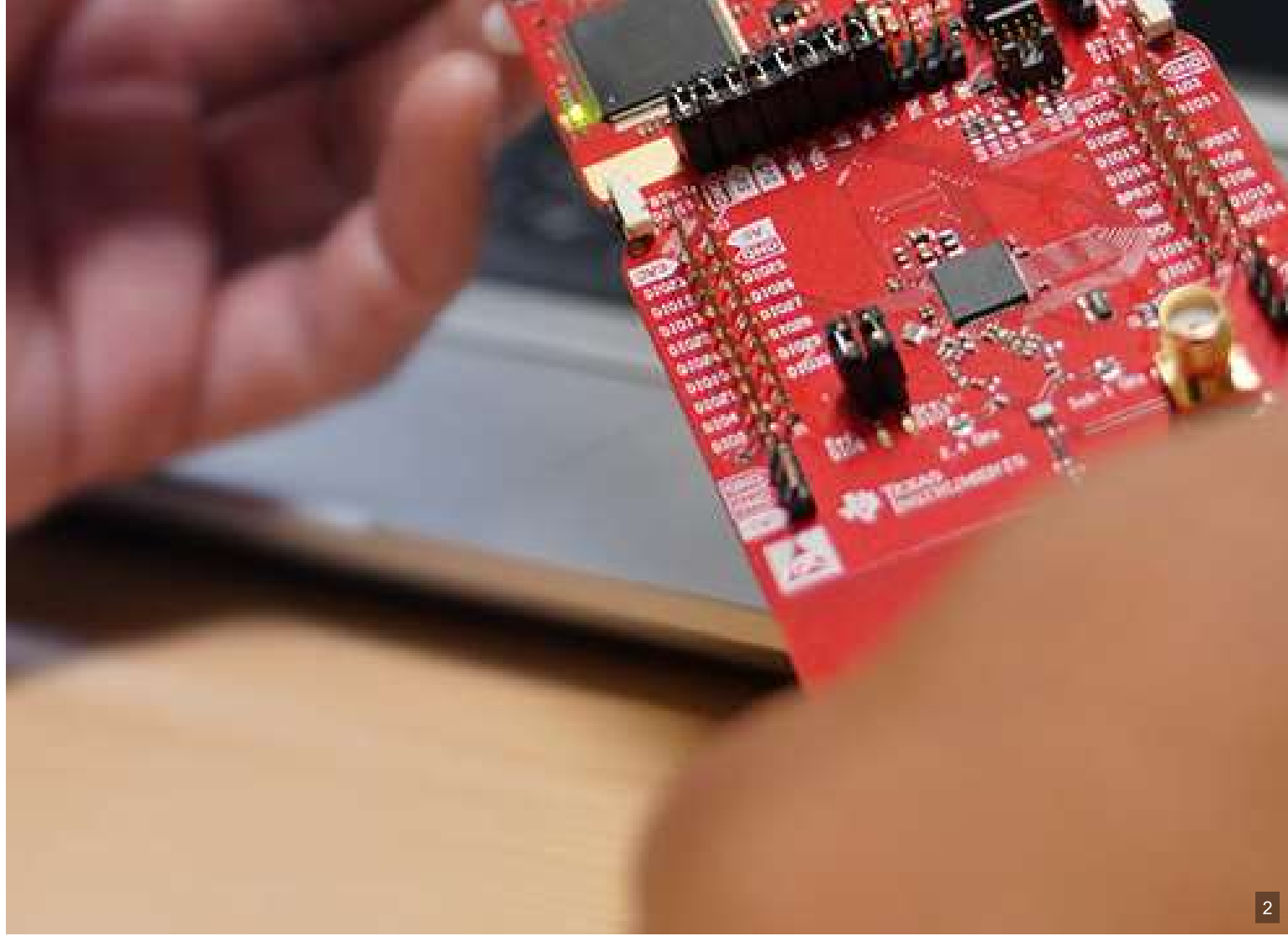






Class 1: An Introduction to Embedded Systems





WHY EMBEDDED SYSTEMS?

What can embedded platforms do?

Examples: (Remember to prepare the final project)

- Laser Scanning Microscope from a Blu-ray Player

(Many projects above are open source)

WHY EMBEDDED SYSTEMS?

What can embedded platforms do?

Examples: (Remember to prepare the final project)

- Laser Scanning Microscope from a Blu-ray Player
- RL Robot Dog

(Many projects above are open source)

WHY EMBEDDED SYSTEMS?

What can embedded platforms do?

Examples: (Remember to prepare the final project)

- Laser Scanning Microscope from a Blu-ray Player
- RL Robot Dog
- Drone Camera

(Many projects above are open source)

WHY EMBEDDED SYSTEMS?

What can embedded platforms do?

Examples: (Remember to prepare the final project)

- Laser Scanning Microscope from a Blu-ray Player
- RL Robot Dog
- Drone Camera
- Remote Control + Remote-Control Car

(Many projects above are open source)

WHY EMBEDDED SYSTEMS?

What can embedded platforms do?

Examples: (Remember to prepare the final project)

- Laser Scanning Microscope from a Blu-ray Player
- RL Robot Dog
- Drone Camera
- Remote Control + Remote-Control Car
- FPV Camera

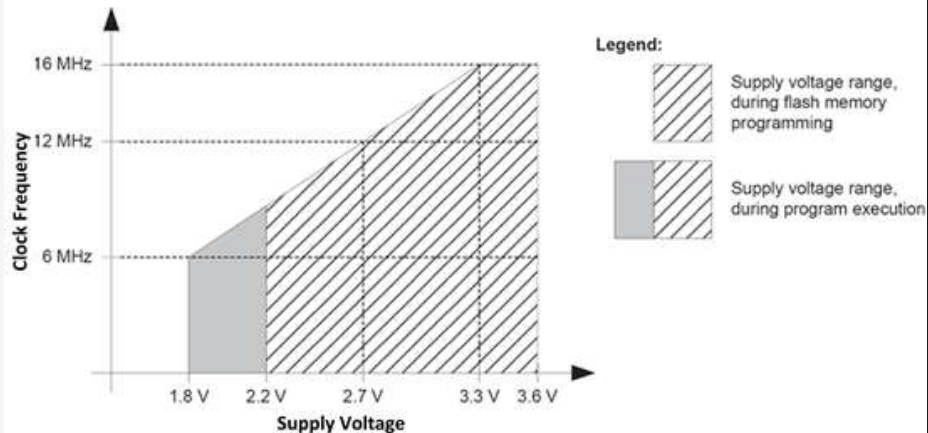
(Many projects above are open source)

WHY MSP430?

- Ultra-low power consumption
- Reducing the impact of EMI
- Extending battery life
- Permanent safety fuse
- Bootloader with 256-bit password protection
- Built-in RTC for time tracking and sleep wake-up

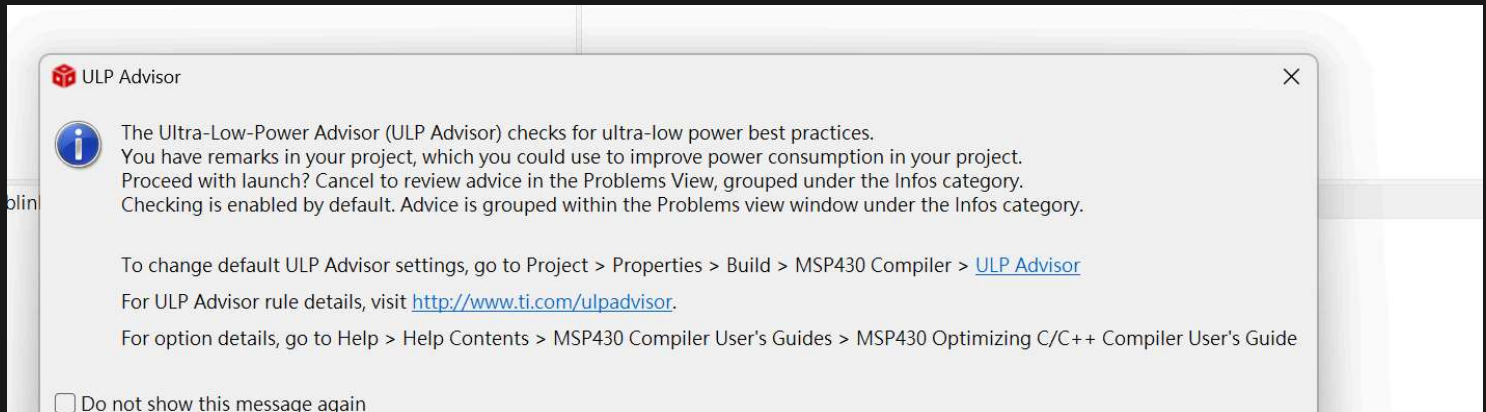
「MSP430 在 3V 系统中以 1MIPS 工作状态下只消耗电流约 $250\mu\text{A}$ ，而且它可以从 $0.8\mu\text{A}$ 的待机状态下在 $1\mu\text{s}$ (F2xx 系列) 内唤醒进入全速运行模式。」 利用超低功耗单片机 MSP430 作为系统伴随芯片 (ti.com)

		MSP430F149 MSP430F1491
Memory	Size	60KB
	Flash	0FFFFh - 0FFE0h
	Flash	0FFFFh - 01100h
Information memory	Size	256 Byte
	Flash	010FFh - 01000h
Boot memory	Size	1KB
	ROM	0FFFh - 0C00h
RAM	Size	2KB
		09FFh - 0200h
Peripherals	16-bit	01FFh - 0100h
	8-bit	0FFh - 010h
	8-bit SFR	0Fh - 00h



DEVELOPMENT ENVIRONMENT: TI CODE COMPOSER STUDIO (CCS)

Refer to : **Link**



The CSS environment compiler helps ensure minimal power consumption, one of its benefits.

WHAT'S EMBEDDED C?

Standard C language is designed for computers with high computational power, meaning those with operating systems, such as Mac, Windows, Linux. However, using it on a microcontroller requires various optimizations. Special syntax includes:

- Declaration of `volatile`: to prevent optimization of variables
- ISR code (#InternalLinks): `__interrupt`

REGISTER OPERATIONS

```
1 To set the pin register to output state, the least significant
2 Any operation |= with 0x01 results in the least significant bit
3 P1DIR      = 0000 0010
4 0x01       = 0000 0001
5 -----
6 Result of OR operation = 0000 0011
7
8 Conversely, ^= with 0x01 calculates an output result that is in
```

Advanced library syntax like DriverLib provides:
GPIO_setOutputHighOnPin()

REGISTER OPERATIONS

```
1 To set the pin register to output state, the least significant
2 Any operation |= with 0x01 results in the least significant bit
3 P1DIR      = 0000 0010
4 0x01       = 0000 0001
5 -----
6 Result of OR operation = 0000 0011
7
8 Conversely, ^= with 0x01 calculates an output result that is in
```

Advanced library syntax like DriverLib provides:
GPIO_setOutputHighOnPin()

REGISTER OPERATIONS

```
1 To set the pin register to output state, the least significant
2 Any operation |= with 0x01 results in the least significant bit
3 P1DIR      = 0000 0010
4 0x01       = 0000 0001
5 -----
6 Result of OR operation = 0000 0011
7
8 Conversely, ^= with 0x01 calculates an output result that is in
```

Advanced library syntax like DriverLib provides:
GPIO_setOutputHighOnPin()

REGISTER OPERATIONS

```
1 To set the pin register to output state, the least significant
2 Any operation |= with 0x01 results in the least significant bit
3 P1DIR      = 0000 0010
4 0x01       = 0000 0001
5 -----
6 Result of OR operation = 0000 0011
7
8 Conversely, ^= with 0x01 calculates an output result that is in
```

Advanced library syntax like DriverLib provides:
GPIO_setOutputHighOnPin()

REGISTER OPERATIONS

```
1 To set the pin register to output state, the least significant
2 Any operation |= with 0x01 results in the least significant bit
3 P1DIR      = 0000 0010
4 0x01       = 0000 0001
5 -----
6 Result of OR operation = 0000 0011
7
8 Conversely, ^= with 0x01 calculates an output result that is in
```

Advanced library syntax like DriverLib provides:
GPIO_setOutputHighOnPin()

INTERRUPT SERVICE ROUTINES (ISR)

Besides event notification, ISRs are used for system operation optimization (multitasking, priority handling).

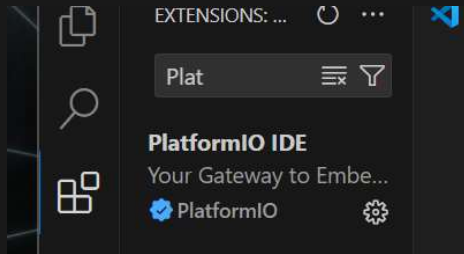
For example, using ISR allows UART transmission without needing to be placed in the main loop. The logic for sending and receiving is encapsulated within

- **【MSP430教學-第十一課】UART串列通訊(六) – TX Interrupt + RX Interrupt 練習 – MSP430系列分享與教學 (wordpress.com)**
- **MSP430 Fundamentals Workshop (ti.com)**

ARDUINO FRAMEWORK

Adds an abstraction layer on top of Embedded C to facilitate ease of use. The development environment utilizes Visual Studio Code with PlatformIO as the editor, stemming from the collaboration with TI to develop the Energia framework.

1. Find the extension by entering "PlatformIO".



2. After downloading, create a new project.

Origin ENERGIA IDE, configuration, compiler or
debugger | TI.com

you can also download .zip and click the

executable file inside

FILE STRUCTURE

- `.ini` file: A PlatformIO project configuration file, specifying the IDE compilation method.

```
[env:lpmsp430g2231]
platform = timsp430
board = lpmsp430g2231
framework = arduino
```

- `src`: Contains the source code, such as `main.cpp`.
- `.pio`: System-generated files, storing temporary files created during compilation and upload processes.
- `.lib`: Stores third-party libraries, including a README file.

Operations: Build, Upload, Monitor, Clean.

TODAY'S TASKS

1. Set up ~~PlatformIO~~(bonus) or energia, **learn to read the official document**
2. Complete the Arduino framework's **LED blinkin** project.
3. Download the CCS development environment.
4. Finish **LED blinking.c**. 

Attachments1: LED blinking in Arduino

```
#include <Arduino.h>

#define LED

//find pins_energia.h for more LED definitions

// initialization and it runs once when you press reset:
void setup() {
    // initialize the digital pin as an output.
    pinMode();
}

// the loop routine runs over and over again forever:
void loop() {
    // turn the LED on and off depends on voltage level
    // 加公頭，每次第1...2...10秒 男捷味開城來城戶
```

Attachments 2: LED blinking in CSS

```
#include <msp430.h>

void main(void)
{
    WDTCTL = ;           // stop watchdog timer ,
    //watchdog用於確認系統響應正常，密碼or控制位
    P1DIR = ;           // configure P1.0 as output

    ;           // 分配一個變量，記得volatile to prevent optimization

    while(1)
    {
        // toggle P1.0
        // delay 使用迴圈
    }
}
```

